



BEGINNER'S GUIDE

Apache Kafka

A short, practical introduction to event streaming — topics, brokers, producers, consumers, and how it all fits together.

Event Streaming

Distributed Systems

Quick Start

CHAPTER 1

What Is Apache Kafka?

Apache Kafka is a **distributed event streaming platform**. Instead of applications talking to each other directly, they publish events (messages) to Kafka, and other applications read those events whenever they need to. Kafka was originally built at LinkedIn and is now one of the most widely used tools for moving data in real time between systems.

THINK OF IT LIKE THIS

Kafka is a durable, high-speed "message bus." Producers drop messages in, consumers pick them up — and Kafka keeps a reliable, ordered log of everything in between.

Why Not Just Use a Database or a Queue?

Traditional queues (like RabbitMQ) usually delete a message once it's consumed, and are optimized for single-consumer delivery. Kafka instead keeps messages for a configurable retention period and lets **many consumers** read the same data independently, at their own pace.

FEATURE	TRADITIONAL QUEUE	APACHE KAFKA
Message lifespan	Deleted after consumption	Retained for a set time (e.g. 7 days)
Multiple consumers	Usually one consumer per message	Many independent consumer groups
Throughput	Moderate	Very high (millions of msgs/sec)
Ordering	Not always guaranteed	Guaranteed within a partition
Replay data	Not possible	Yes — re-read from any offset

Common Use Cases

Real-time analytics

Stream clickstream or sensor data into dashboards as it happens.

Microservice messaging

Decouple services so they don't call each other directly.

Log aggregation

Collect logs from many services into one central pipeline.

Event sourcing

Store every state change as an event for audit and replay.

CHAPTER 2

Core Concepts

Before touching any code, it helps to know Kafka's vocabulary. These five terms cover almost everything you'll encounter as a beginner.

KEY TERMS

Topic

A named category of messages, similar to a table name — e.g. "orders" or "page_views".

Partition

Each topic is split into partitions so it can be spread across machines and read in parallel. Order is only guaranteed within a single partition.

Broker

A Kafka server that stores data and serves client requests. A Kafka cluster is made of several brokers.

Producer

An application that writes (publishes) messages to a topic.

Consumer

An application that reads (subscribes to) messages from a topic.

Topics and Partitions, Visually

Topic: orders

Partition 0

Partition 1

Partition 2

Each partition is an ordered, append-only log. New messages are always added to the end.

NOTE

More partitions = more parallelism, but also more overhead. A good starting point for learning is 3 partitions per topic.

Brokers and Clusters

A Kafka cluster is a group of brokers working together. Each partition is stored on one broker as the "leader," and can be copied to other brokers as "replicas" for fault tolerance — more on that in Chapter 5.

CHAPTER 3

How Messages Flow

Once you understand topics and partitions, the message flow is simple: producers write, Kafka stores, consumers read.



Writing a Message (Producer Side)

A producer sends a message with an optional **key** and a **value**. If a key is provided, Kafka uses it to decide which partition the message lands in — messages with the same key always go to the same partition, which keeps their order.

PRODUCER.PY

```
# Simple Kafka producer example
from kafka import KafkaProducer
import json

producer = KafkaProducer(
    bootstrap_servers='localhost:9092',
    value_serializer=lambda v: json.dumps(v).encode('utf-8')
)

# Send a message with a key so all events for this order stay ordered
producer.send('orders', key=b'order_1001', value={'status': 'created'})
producer.flush()
```

Reading a Message (Consumer Side)

A consumer connects to the cluster, subscribes to one or more topics, and continuously polls for new messages.

CONSUMER.PY

```
# Simple Kafka consumer example
from kafka import KafkaConsumer
import json

consumer = KafkaConsumer(
    'orders',
    bootstrap_servers='localhost:9092',
    value_deserializer=lambda v: json.loads(v.decode('utf-8'))
)

for message in consumer:
    print(f"Received: {message.value}")
```

TIP

Kafka doesn't remove messages after they're read. Consumers just keep track of their own position — this is what makes replaying data possible.

CHAPTER 4

Consumer Groups & Offsets

Two ideas make Kafka's consumption model powerful: **offsets** and **consumer groups**.

Offsets

Every message in a partition gets a sequential ID called an offset (0, 1, 2, ...). A consumer simply remembers the last offset it processed, so it can pick up exactly where it left off — even after a restart.

Consumer Groups

Consumers are organized into **consumer groups**. Kafka automatically spreads a topic's partitions across the consumers in a group, so each partition is only read by one consumer in that group at a time. This gives you parallel processing without any extra coordination code.

Topic "orders" (3 partitions) — Consumer Group "billing-service"

Partition 0 → Consumer A

Partition 1 → Consumer B

Partition 2 → Consumer C

COMMON PITFALL

If you add more consumers to a group than there are partitions, the extra consumers sit idle. Partition count sets the upper limit on parallelism within one group.

Different consumer groups are fully independent of each other — a "billing-service" group and an "analytics-service" group can both read the same topic from their own separate offsets, without affecting one another.

CHAPTER 5

Replication & Fault Tolerance

Kafka is designed to keep running even when individual machines fail. It does this through **replication**.

Leaders and Replicas

Each partition has one broker acting as the **leader**, which handles all reads and writes for that partition. Other brokers hold **replicas** — copies that stay in sync with the leader. If the leader broker goes down, one of the in-sync replicas is automatically promoted to leader.

Partition 0 — Replication Factor 3

Broker 1 (Leader)

Broker 2 (Replica)

Broker 3 (Replica)

NOTE

A replication factor of 3 is a common default in production — it tolerates the loss of up to two brokers without losing data.

Durability Settings

Producers can choose how strict they want to be about durability using the **acks** setting:

ACKS VALUE	BEHAVIOR	TRADE-OFF
0	Fire and forget	Fastest, risk of data loss
1	Leader confirms write	Balanced default
all	All in-sync replicas confirm	Safest, slightly slower

CHAPTER 6

Getting Started Locally

You don't need a cluster to learn Kafka — a single-node setup on your laptop is enough to try everything in this guide.

1. Start Kafka (Docker)

TERMINAL

```
# docker-compose.yml with a single broker + ZooKeeper/KRaft
# then simply run:
docker compose up -d
```

2. Create a Topic

TERMINAL

```
kafka-topics.sh --create \
  --topic orders \
  --bootstrap-server localhost:9092 \
  --partitions 3 \
  --replication-factor 1
```

3. Send and Read Messages from the CLI

TERMINAL

```
# In one terminal – produce messages
kafka-console-producer.sh --topic orders --bootstrap-server localhost:9092

# In another terminal – consume messages
kafka-console-consumer.sh --topic orders --from-beginning --bootstrap-server
localhost:9092
```

TIP

Type a few lines into the producer terminal and watch them instantly appear in the consumer terminal — it's the fastest way to build intuition for how Kafka moves data.

CHAPTER 7

Where to Go Next

This guide covers the fundamentals — enough to understand how Kafka works and run a first local example. From here, the natural next steps are:

Kafka Connect

Plug Kafka into databases and other systems without custom code.

Schema Registry

Enforce message structure using Avro or Protobuf schemas.

Kafka Streams / ksqldb

Process and transform data as it flows through topics.

Monitoring

Track consumer lag and broker health in production.

RECAP

Producers write to topics → topics are split into partitions across brokers → consumer groups read those partitions in parallel → replication keeps data safe if a broker fails.